

CO3_AT1

Java Programs CSA0911

AL NAJEEB A / 192424325 / SLOT B

Q1 try-with-resources Misuse

```
Q1 try-with-resources Misuse 25 marks

import java.io.*;

public class TryWithResourcesDemo {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(new FileReader("data.txt"));
        try (reader) {
            String line = reader.readLine();
            System.out.println(line);
        } catch (IOException e) {
            e.printStackTrace();
        }
        try (FileWriter writer = new FileWriter("out.txt")) {
            writer.write("Hello");
        }
        class NotCloseable {
            void write() {}
        }
        try (NotCloseable nc = new NotCloseable()) {
            nc.write();
        }
        try (FileReader fr1 = new FileReader("a.txt");
            FileReader fr2 = new FileReader("b.txt")) {
            fr1.close();
            fr2.close(); close (double-close, though usually harmless it's a logic bug/anti-pattern here compounded by bug 5)
        } catch (IOException e) {
        } finally {
            fr1.close();
        }
    }
}
```

```
import java.io.*;
public class Main{
    public static void main(String[] args){
        try(FileWriter writer = new FileWriter("out.txt")){
            writer.write("Hello World");
            System.out.println("File written successfully.");
        }
        catch(IOException e){
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT

Output

File written successfully.

Q2 Custom Exception Class

Q2 Custom Exception Class 25 marks

```
public class CustomExceptionDemo {
    public static void main(String[] args) {
        try {
            validateAge(-5);
        } catch (InvalidAgeException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    static void validateAge(int age) {
        if (age < 0) {
            throw InvalidAgeException("Age cannot be negative");
        }
    }

    class InvalidAgeException extends Exception {
        public InvalidAgeException(String msg) {
        }

        public String getMessage() {
            return "Custom message";
        }

        public void printError() {
            System.out.println("Error occurred");
        }
    }
}
```

```
class InvalidAgeException extends Exception{
    public InvalidAgeException(String msg){
        super(msg);
    }
}

public class Main{
    static void validateAge(int age) throws InvalidAgeException{
        if(age < 0){
            throw new InvalidAgeException("Age cannot be negative");
        }
        System.out.println("Valid age: " + age);
    }

    public static void main(String[] args){
        try{
            validateAge(-5);
        }
        catch(InvalidAgeException e){
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT

Output

Error: Age cannot be negative

Q3 wait() and notify() Coordination

```
wait() and notify() Coordination 25 marks

public class WaitNotifyDemo {
    private boolean dataReady = false;
    private final Object lock = new Object();

    public void produce() {
        synchronized (lock) {
            dataReady = true;
            System.out.println("Data produced");
            lock.notify();
        }
    }

    public void consume() {
        if (!dataReady) {
            lock.wait();
        } catch (InterruptedException e) {
        }
        System.out.println("Data consumed");
    }

    public void consumeCorrectly() {
        synchronized (lock) {
            if (!dataReady) {
                lock.wait();
            } catch (InterruptedException e) {
                Thread.interrupted();
            }
        }
    }

    public static void main(String[] args) {
        WaitNotifyDemo demo = new WaitNotifyDemo();
        Thread producer = new Thread(demo::produce);
        Thread consumer = new Thread(demo::consumeCorrectly);
        producer.start();
        consumer.start();
    }
}
```

```
public class Main{
    static boolean dataReady = false;
    static Object lock = new Object();

    public static void main(String[] args){

        Thread producer = new Thread(new Runnable() {
            public void run() {
                synchronized(lock){
                    dataReady = true;
                    System.out.println("Data produced.");
                    lock.notify();
                }
            }
        });

        Thread consumer = new Thread(new Runnable() {
            public void run() {
                synchronized(lock){
                    while(!dataReady){
                        try{
                            lock.wait();
                        }
                        catch(InterruptedException e){
                            e.printStackTrace();
                        }
                    }
                    System.out.println("Data consumed.");
                }
            }
        });

        consumer.start();
        producer.start();
    }
}
```

OUTPUT

Output

```
Data produced.
Data consumed.
```